Russia-Ukraine War Fatality Analysis

Problem Setting

On February of 2022, Russia began its full-scale invasion of Ukraine, fully igniting a war that's been very costly on both sides in terms of manpower and artillery. It's often tricky for military leaders to decide what vehicles and equipment to distribute throughout the battlefield for maximum effectiveness. We are interested in predicting the Ukraine war casualties resulting from the deployment of various Russian military equipment. Countries could benefit from this analysis because it could inform them of what weapons or vehicles they should invest more in for their own armed conflicts as geopolitical tensions rise between nations with considerable military presences. Additionally, modeling to predict casualties based on troop/equipment deployments allows the military to predict outcomes for battles with greater accuracy, allowing for more robust strategic planning.

Problem Definition

With tabular data representing the equipment and the casualties from the war, we're interested in which equipment has the largest impact on casualties and whether casualties change from when certain equipment is deployed. For example, do we see more personnel losses on Russia's side or Ukraine's when more helicopters are deployed? How many more casualties are there on Ukraine's side when Russia brings in more tanks? Does the deployment aircraft and helicopters from Russia result in casualties for either side? Also, how long does it take for equipment/ personnel deployments to have an effect on casualties?

Data Sources

The main data set that inspired this exploration was posted on Kaggle by a developer from Ukraine looking to create more awareness about the war. He updates the dataset weekly using multiple online sources. We plan to supplement his data set with a related dataset that details the locations of the casualties, military factions, along with other details about the war. This supplementary data set was updated 10 months ago.

Another supplementary dataset is from ACLED. ACLED is a widely used political violence and protest data around the world. Unlike Kaggle, you can custom query the data instead of browsing for premade datasets. It's a remote non-profit that monitors conflicts to provide real-time analysis.

Data Overview

The personnel loss data set has five columns with 1,445 rows. Out of the box, it has two object columns, two integer columns, and one float column. The only missing values from this data frame are from the pow column, which has 1,383 missing values. This likely means that there were no prisoners of war during those entries. The earliest entry for this data frame was February 25th, 2022 and the most recent entry is February 8th, 2026. The personnel* column seems to

indicate the estimation of the corresponding personnel entry, as the unique values under that column are “about” and “more”

The equipment loss data set has 1,446 rows and 19 columns that detail the cumulative losses in Russian military equipment. There’s a couple columns for dates and total number of days since the start of the war. There are numerical columns for aircraft equipment, ground vehicles, artillery weapons, naval vehicles, and support equipment. There’s also a column indicating which operational front suffered the most losses from that day.

The equipment correction dataset is very small and only represents some minor corrections to the equipment dataset.

The ACLED dataset has 164,187 rows and 33 columns. It includes dates of the conflicts and classifies them by disorder type and event type. The actor columns indicate the attacking parties and the parties that were attacked. There’s location data including the exact coordinates of the conflicts and the name of the places, which could be used to map out the conflicts of the war. Lastly, there’s numerical and nominal data for the impact of those conflicts, which includes the fatalities, the local population, and summaries of the event that took place.

Data sources:

ACLED custom query:

https://acleddata.com/conflict-data/data-export-tool

Russia_losses_personnel.csv,

russia_losses_eqipment_correction.csv, and

russia_losses_equipment.csv:

https://www.kaggle.com/datasets/piterfm/2022-ukraine-russian-war?select=russia_losses_personnel.csv

Data Cleaning, Dimension Reduction, Variable Selection

We start by analyzing the 4 data files, looking for data types, column names, NaNs, and feature types. After spending some time with the data and gathering domain knowledge, we made several decisions on how to best organize the data for processing without losing important signals. Many of the Nans were supposed to be 0s and some of the columns provided complex semantic information in the form of text annotations, and some of the columns were redundant or provided information specific to the data. For example, geo_precision does not help us, since it only refers to how precise the estimate of the battle location was.

Since the ACLED was very large, we had to drop or encode many of its features. All columns relating to location were dropped except for the coordinates. We focused on classifying the actors from the conflicts by using regex matching to label which side the actors and

associated actors were on. For the actors that didn't say "Russia" or "Ukraine" in their names, we labeled them as either militia or other, transformed to integers, then one hot encoded them. For other categorical variables like disorder types, event types, actor affiliation, and source scales, we also used one-hot encoding. Civilian targeting was encoded in 1s and 0s. The population data was cleaned via the removal of missing values. Since there was already a column for the event date, we removed the columns for year and time precision.

The cleaning for the personnel loss data was more straightforward since it was mainly on dealing with the missing data and trimming down the structure. The personnel* column was only an indicator of the estimate of fatalities, so we removed it. The POW column had many missing values so we imputed them as 0s. The cleaning for the equipment loss data was more comprehensive. Several columns for the equipment types had null values which we imputed as 0s. We dropped the greatest loss direction column because it didn't add anything to the quantitative analysis of the data.

ACLED data

- Got rid of all of the location data for the acled data except for the coordinates.
- created a new column based on the actor1 column that indicated which side of the war (Ukraine, Russia, or Neither) generally the attacking party was on.
- Encoded civilian targeting to 1 or 0
- Drop column for admin 1, admin 2, admin 3, tags,
- Drop Nas for population best but keep column
- Regex match for Associated actor 1, assoc actor 2
- Regex match for actor 1, actor 2 if they say Russia or Ukraine in the name
- Drop year, drop time precision
- one hot encode disorder_type
- one hot encode event_type
- one hot encode sub_event_type
- one hot encode actor1, assoc_actor_1, inter1, actor2, assoc_actor_2, inter2, interaction
- drop iso, region, country, location
- drop geo_precision
- one hot encode source, source_scale
- drop notes
- drop date, keep event date, drop timestamp

Personnel loss data frame

- drop the column personnel*
- impute the missing values under the pow column as zeros.
- Drop day column since we already have date

Equipment loss

- encode NaNs from military auto, fuel tank, special equipment, mobile, vehicle and fuel tank, cruise missiles, submarines, to 0
- drop greatest loss direction col
- Drop day column since we already have date

Merged Data

- Merge a left join on the ACLED data on date, to preserve multiple events per day
- Drop NA's handle with 0 impute or drop when there is no population recorded
- One hot-encode nominal categorical variables
- Log transform and scale data after isolating the target variable
- Results in numerically balanced features, not so different in this case with or without scaling due to the sparsity of some events

Note fatalities as well as all equipment are reported very sparsely and likely are underestimations; fatalities has a mean of 1, a standard deviation of 7.69 and a max of 600 out of 164,186 observations. Equipment deployment is similarly rare, likely implying large battles.

Data Manipulation

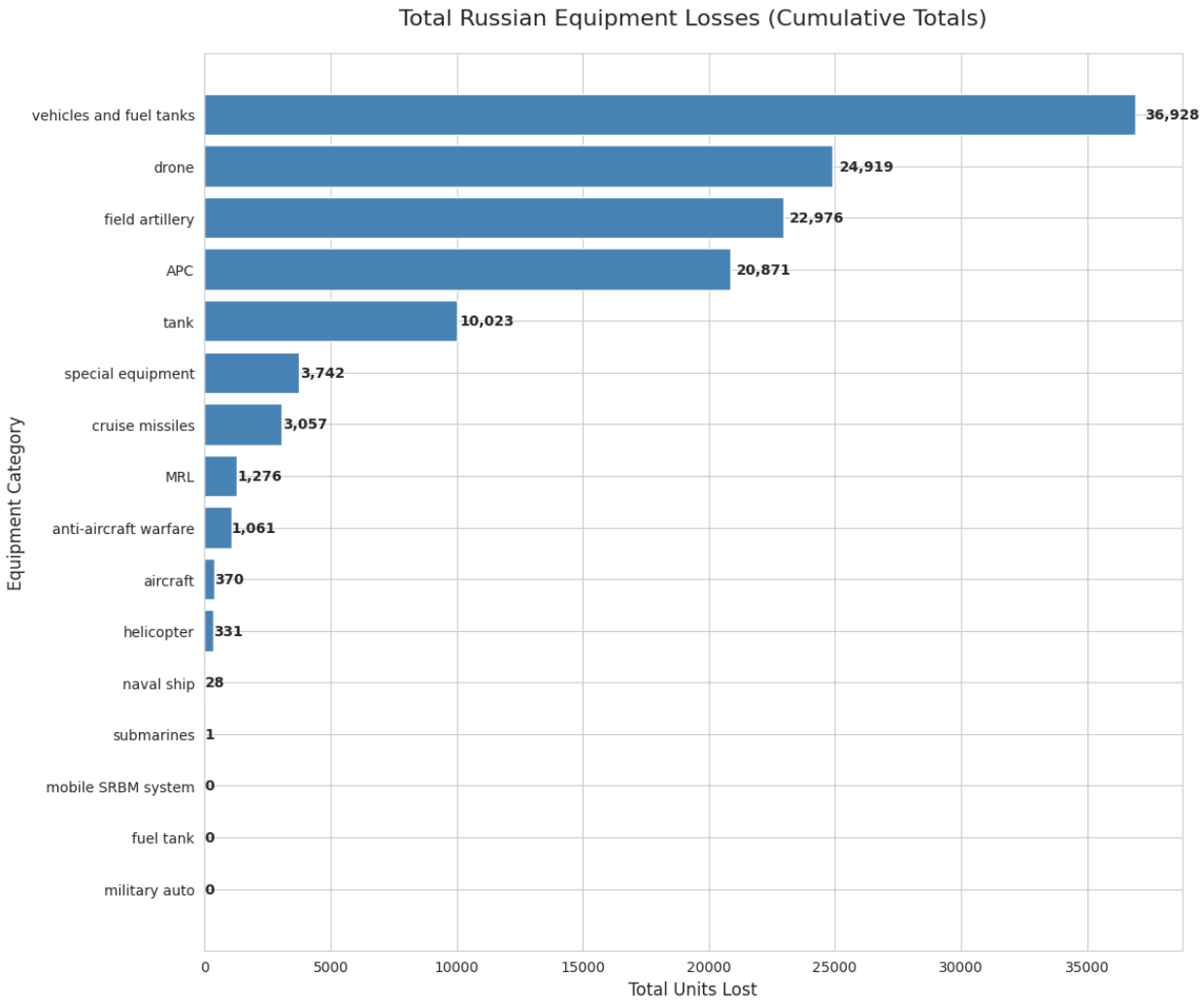
In our code, we approached preprocessing the data frames separately first, handling the columns we want to include as well as trying to rationalize how to handle missing values based on domain knowledge. Most of the preprocessing to clean the data is done before merging the data. When we merge, we left join the three datasets onto the date of the ALCED data, which lists multiple events per day; this way we preserve the multiple events that happen per day. After merging, the data is once again checked for NAs and columns that are no longer needed are dropped.

Then variables are encoded, for example we encode the actors field numerically to indicate Ukraine, Russia, etc. Then we one hot encode those variables which are categorical and nominal. After encoding and ensuring numeric data types for all features, the data is log transformed, then it is standard scaled to unit variance. Note when scaling, we are only scaling

the data separate from the target variable y = fatalities, which is later reintroduced to the data. This ensures no data leakage from the target variable to the entire data set during transformation. We separately save the scaled and unscaled data. In testing, both perform very similar, in this case scaling did not do much to transform the data when conducting our analysis. This is likely due to the extreme sparsity of the data and the underreporting of fatalities, leading to class imbalances. Our final data frame has been reconstituted from one hot encoding after scaling, so we end up with 21 features + date index + target column.

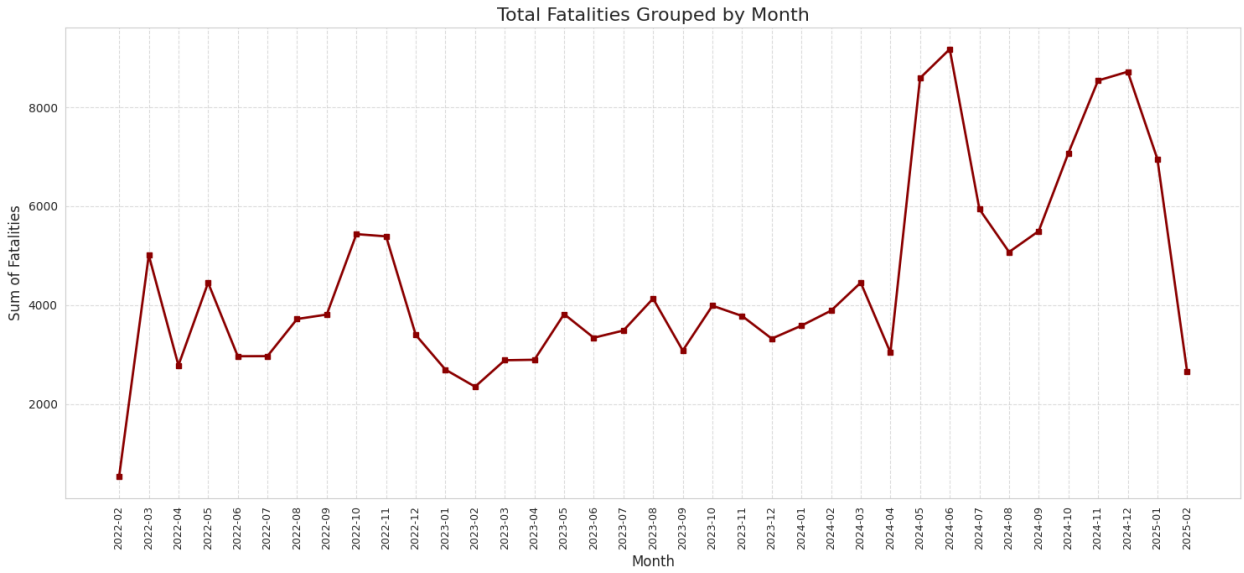
Data Visualization and Observations

In this section we highlight some of the most interesting observations, associations, and correlations between the various features in our data. These are our initial explorations and findings with visualizations and explanations for each visualization:



The top four equipment categories that suffered the most losses have been vehicles/fuel tanks, drones, and field artillery.

Based on the Russian equipment losses, the country has been heavily focused on ground logistics, drones and artillery weapons as its main way of wearing down Ukraine. The vehicles and fuel tanks make up most of the losses. This implies that the Russians have been mostly targeting Ukraine's logistics and supply chains. The high number of drone losses puts a spotlight on how important unmanned weapons have been in modern warfare when it comes to conducting recon and strike operations along with conserving troops. The lack of losses in aircraft and naval equipment reinforce the idea that this war is mainly fought on ground.

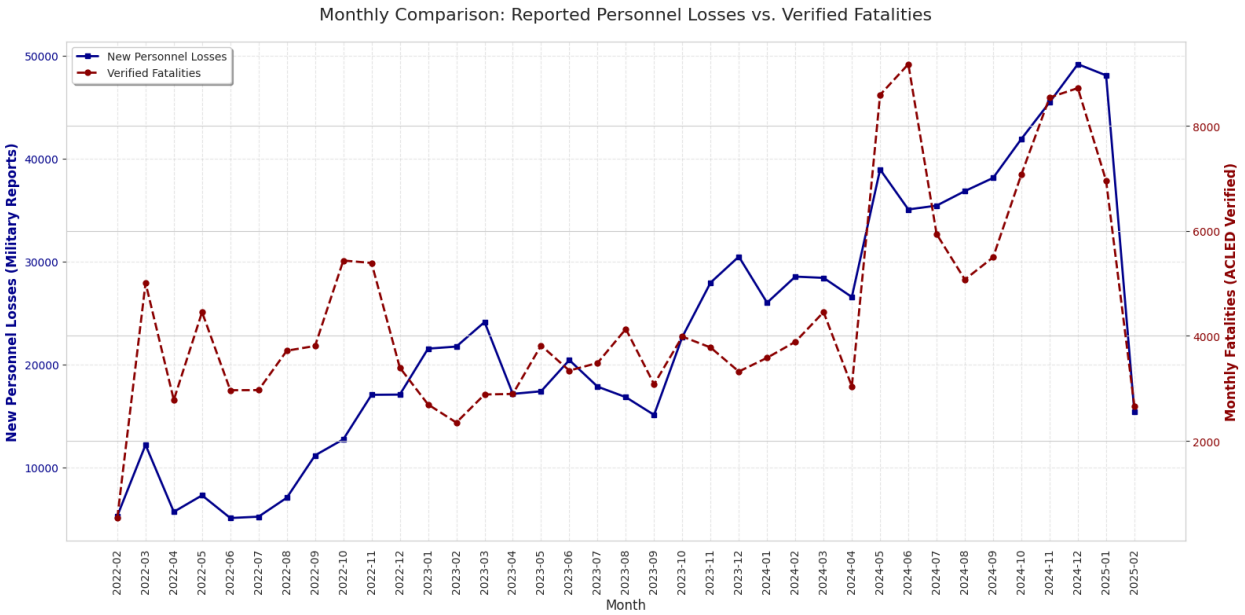


The ACLED's fatality reports indicate spikes during early 2022, mid 2024, and late 2024.

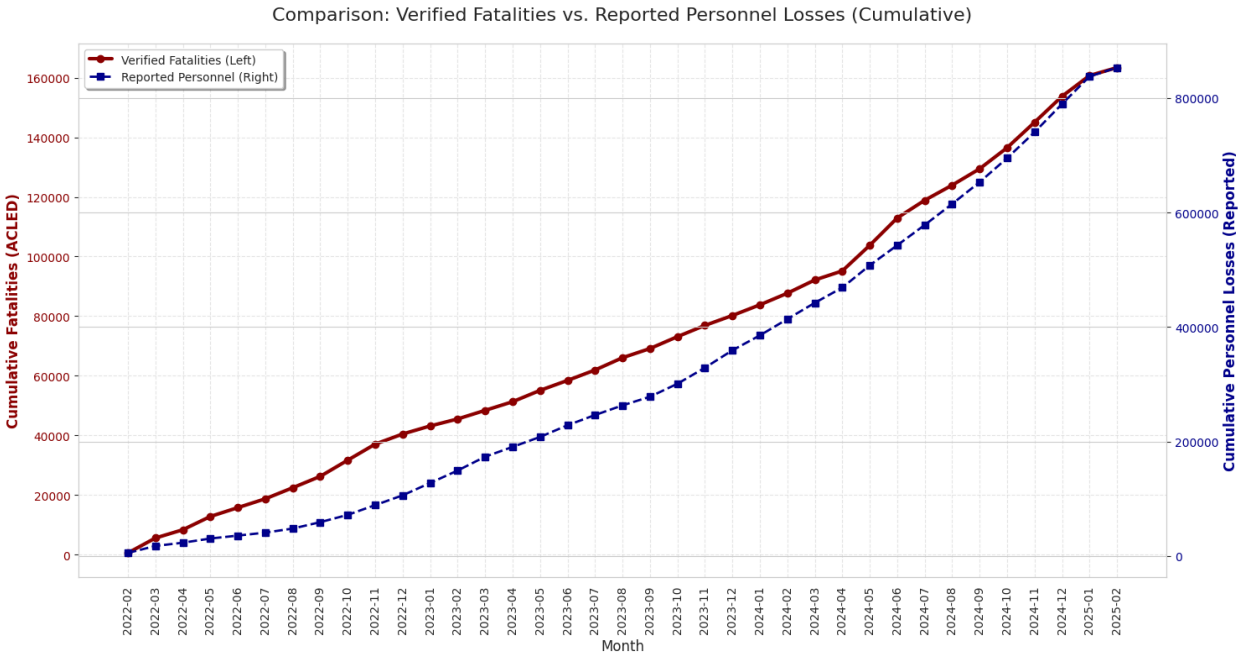


Russia personnel loss reports indicate spikes during early 2023, late 2023, mid 2024, and late 2024.

The largest spike was in late 2024, when both sides intensified their operations. The multiple spikes on those charts indicate cyclical surges, which illustrate the evolving tactics from either side over time.

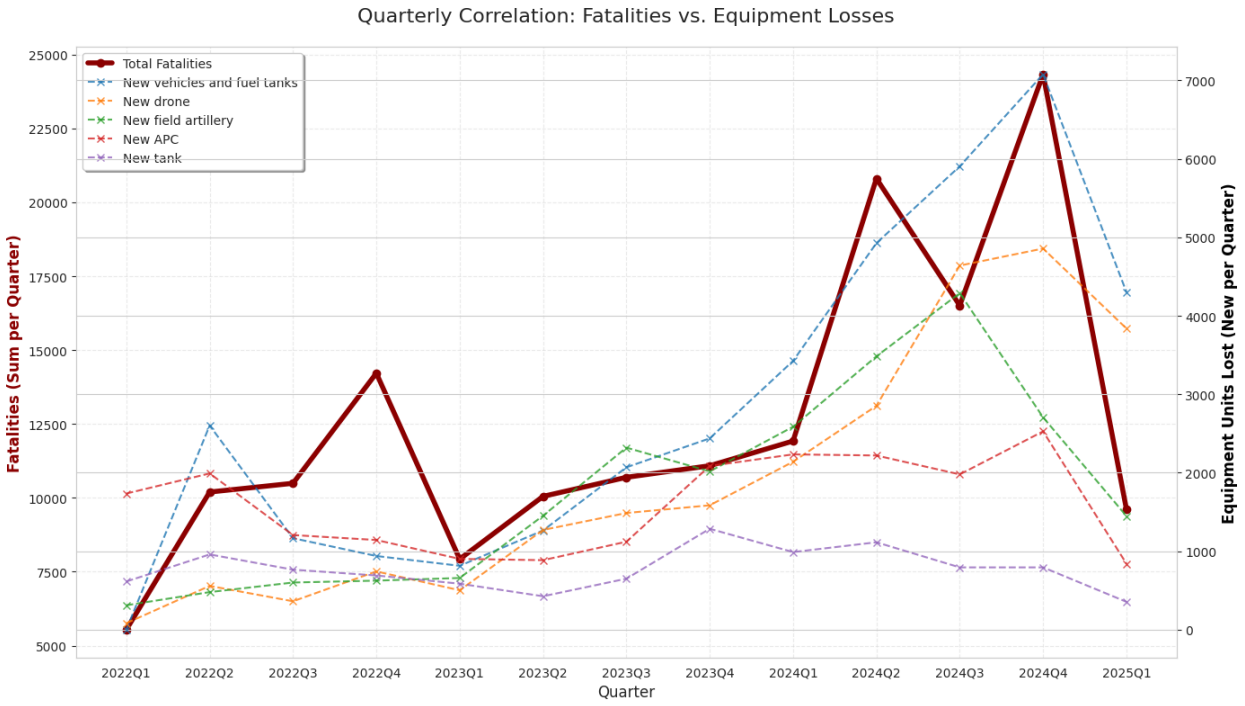


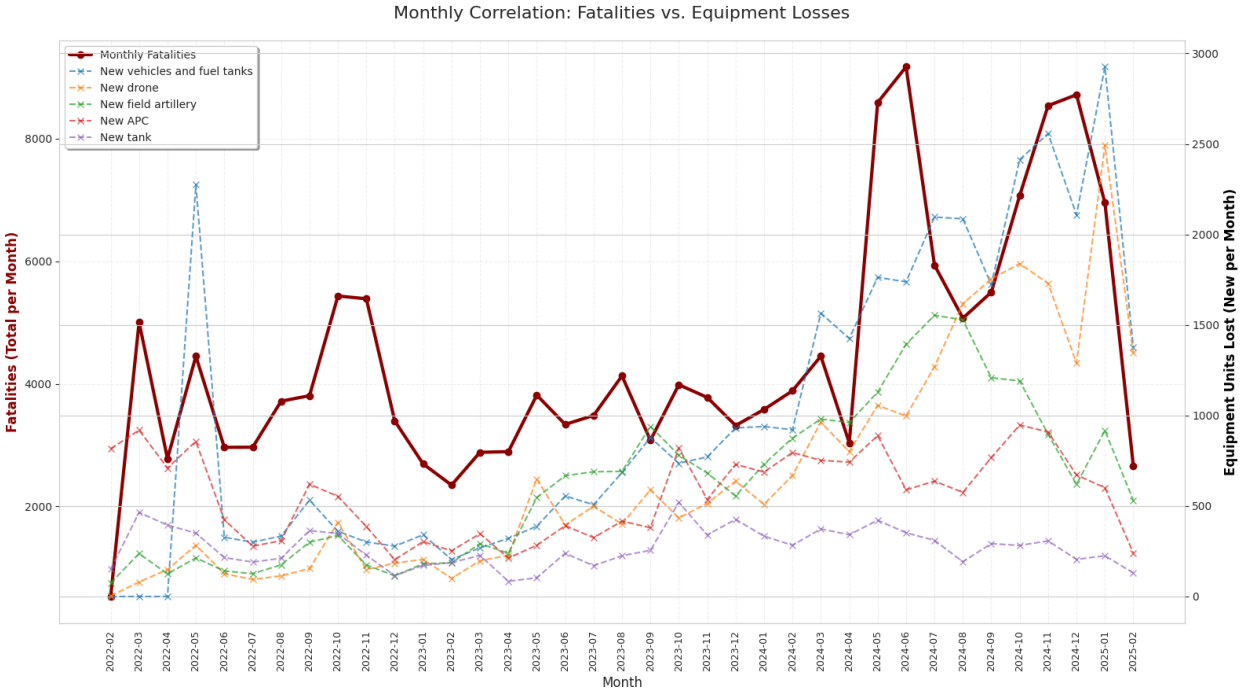
After matching the scales, the proportions of the fatalities and Russian personnel losses seem to match after 2022.



After matching the scales, the proportions of the cumulative fatalities and Russian personnel losses seem to match.

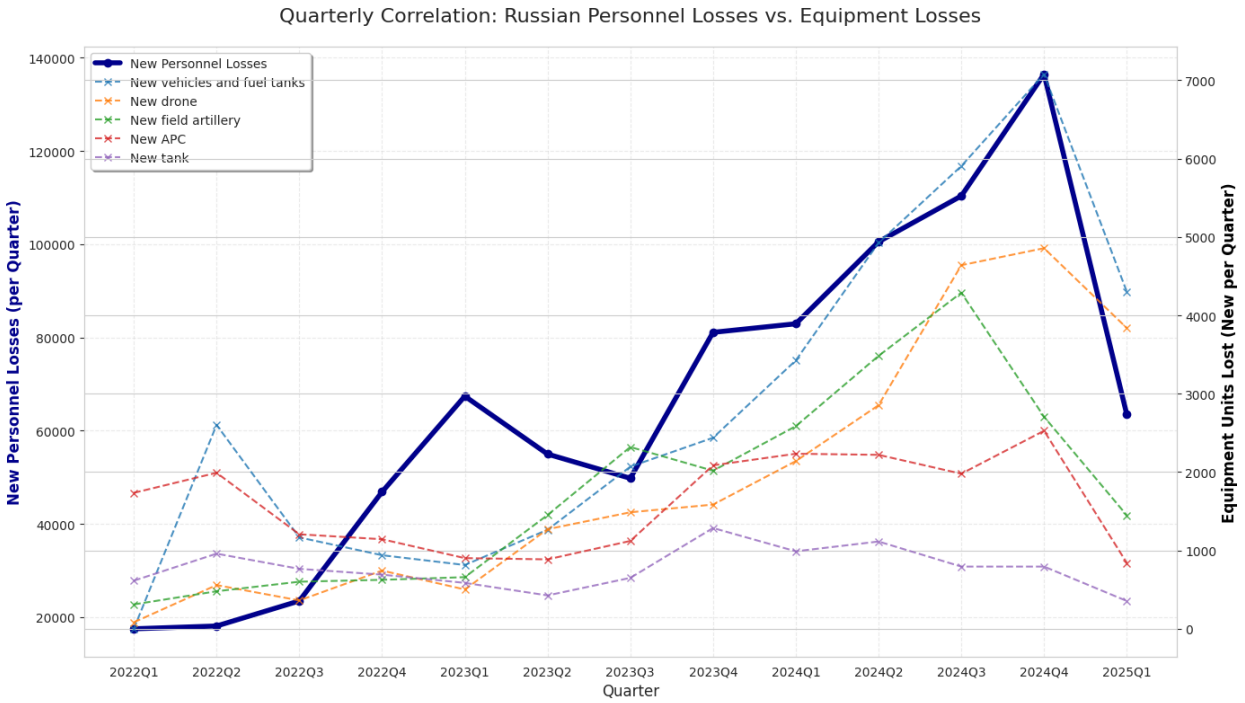
After the volatility from early 2022, the monthly spikes and declines for both the acled data and the Russian data increasingly mirror each other over time. The cumulative chart reinforces this idea with both charts being almost parallel. This indicates how, despite the scales of losses, there is a strong level of proportionality from the way the losses are being reported from both sources.

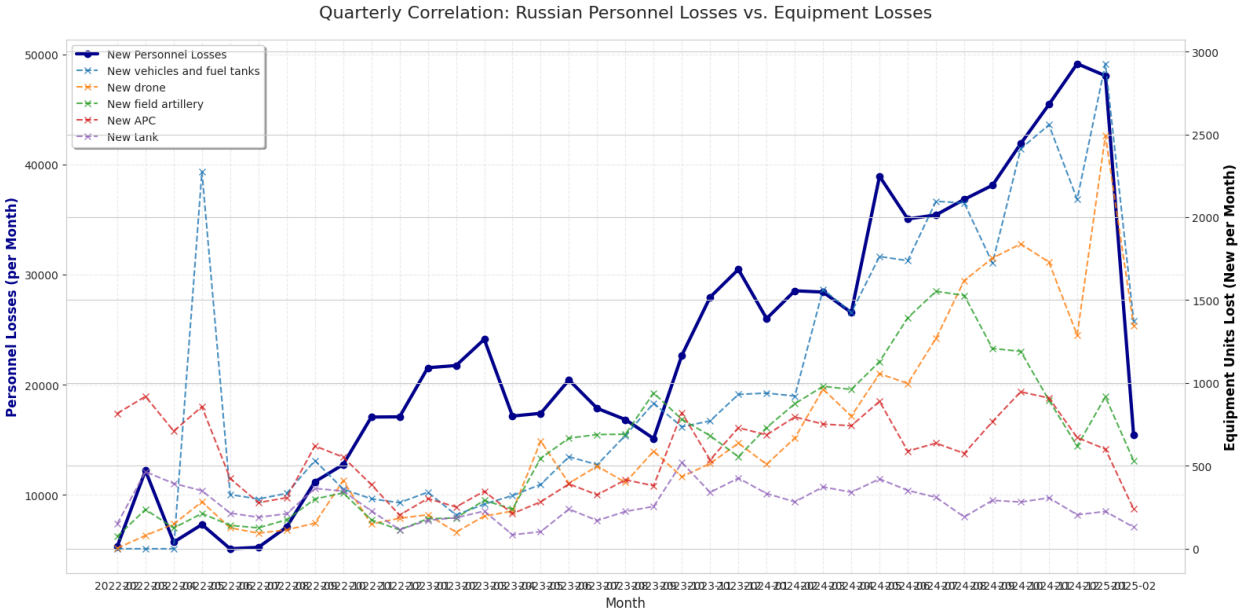




Both the quarterly and monthly charts for fatality and Russian equipment losses show a clear

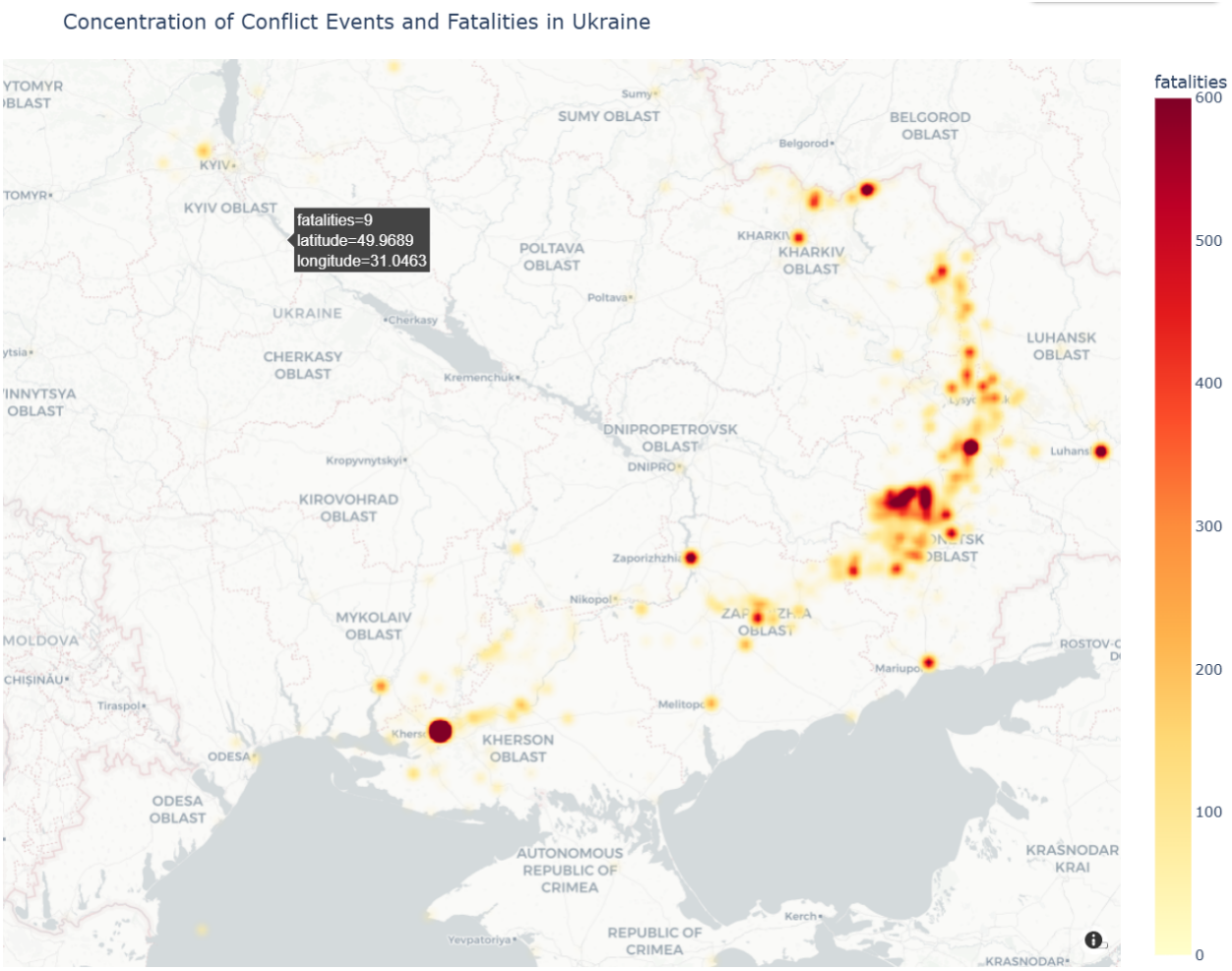
relationship between them. The quarterly chart illustrates the spikes in fatalities from Q4 of 2022 and throughout 2024 closely aligning with the increases in losses of vehicles, artillery, drones and armored platforms. This makes sense because that's what happens when one side pours more resources into their ground operations against the other. The short-term surges in fatalities from the monthly chart support this idea from how often it coincides with the spikes in vehicle and artillery losses.



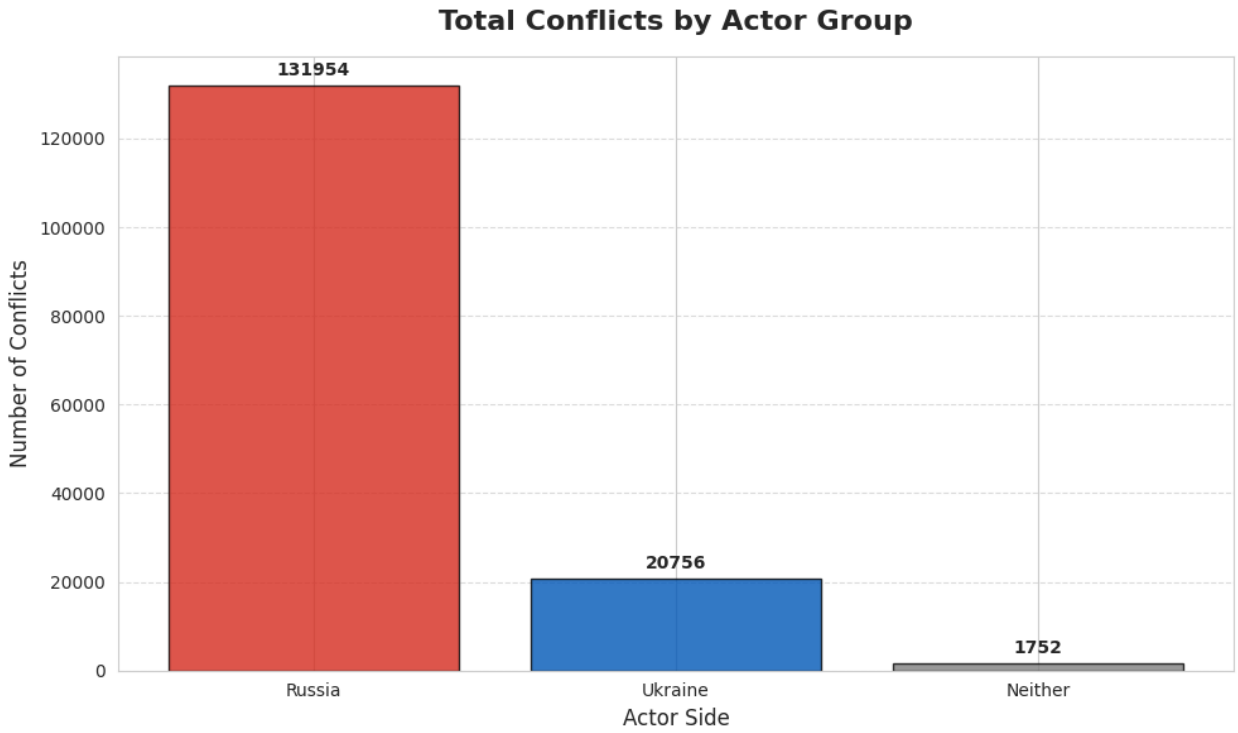


Just like the charts on the fatality and Russian equipment loss, these two charts show a strong

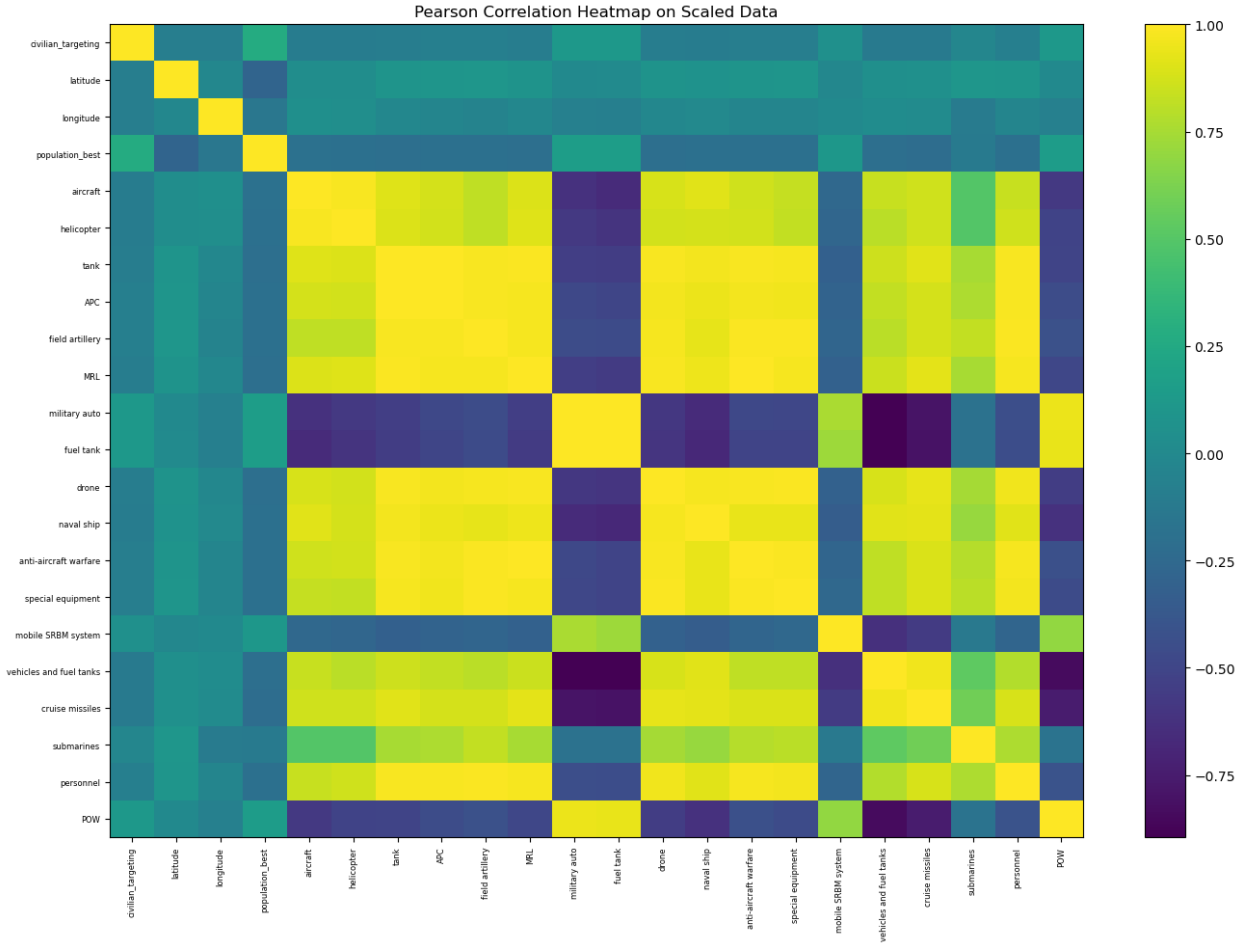
positive relationship between the Russian personnel and equipment losses. The sharp increases in personnel losses, especially in late 2023 and throughout 2024, lineup with the spikes in vehicles, drones, and field artillery losses. This alignment was the most pronounced in 2024, when both personnel and equipment losses reached there highest levels before declining the next year.



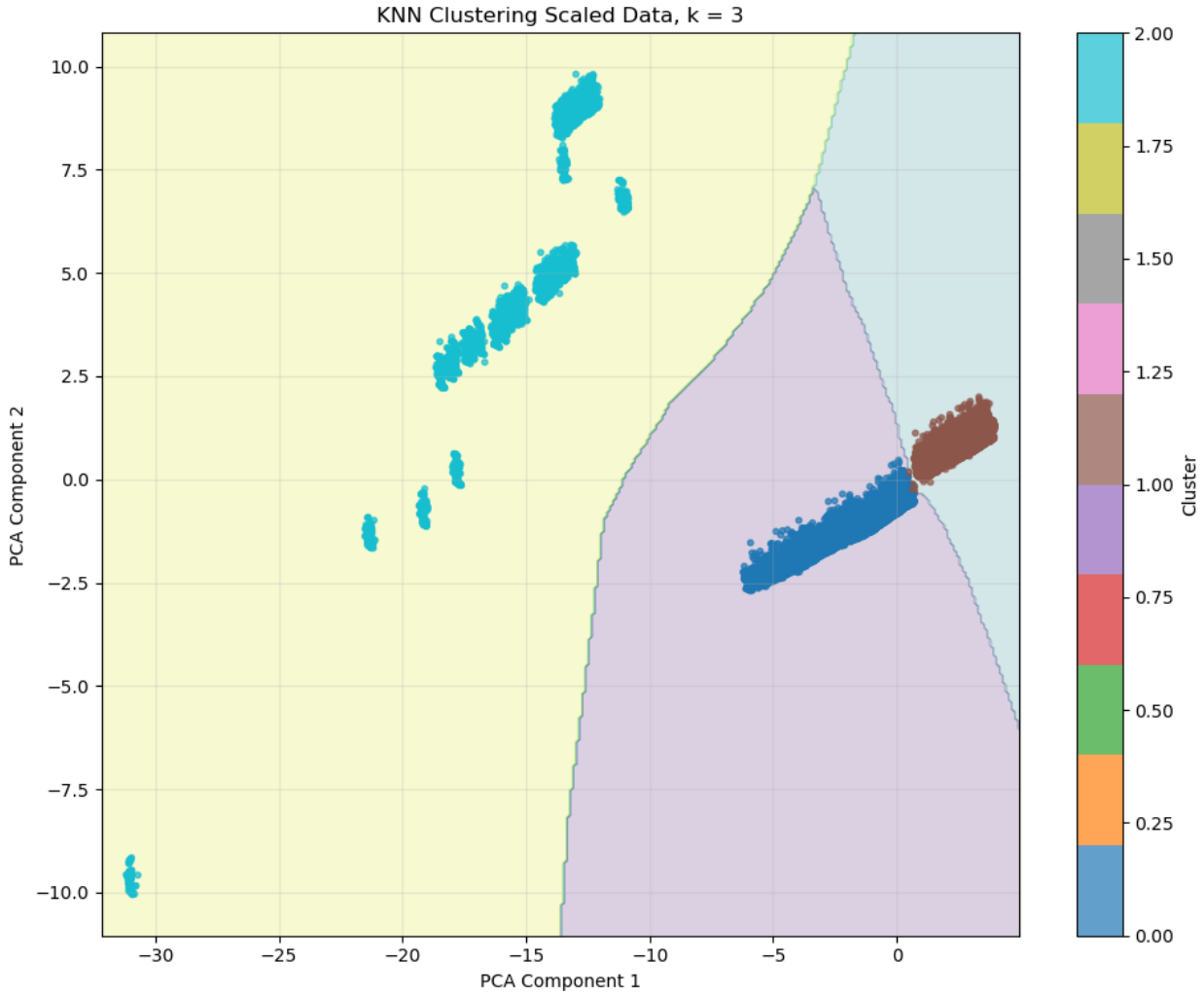
This heatmap of the fatalities in Ukraine displays where most many of the conflicts took place, which was along Eastern and Southern Ukraine. The regions where the most people have died are along the Donbas region and Southern cities. Central and Western Ukraine have significantly less fatalities, showing how the war is concentrated along the Eastern and Southeastern areas of the country. When you line up the locations the clusters, you can see how the war is concentrated on the front lines as opposed to scattered, nationwide attacks.



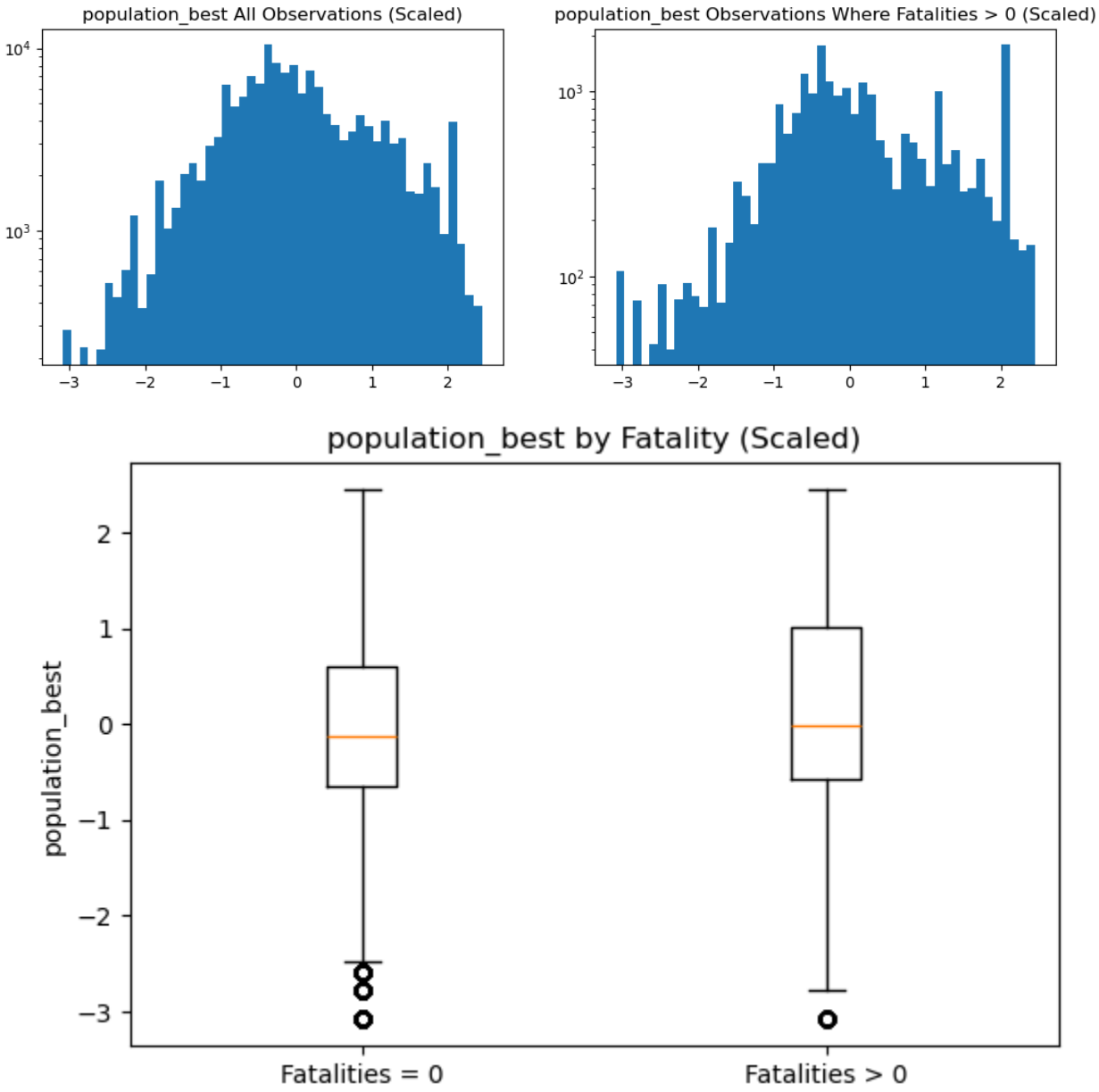
According to this chart, Russia has incited an overwhelming majority of the conflicts throughout the war, with about six times as many as Ukraine. It really goes to show how Ukraine is the underdog for this whole war. It's also impressive that Ukraine has been able to last for this long with the territories of the war not changing much since the start of it.



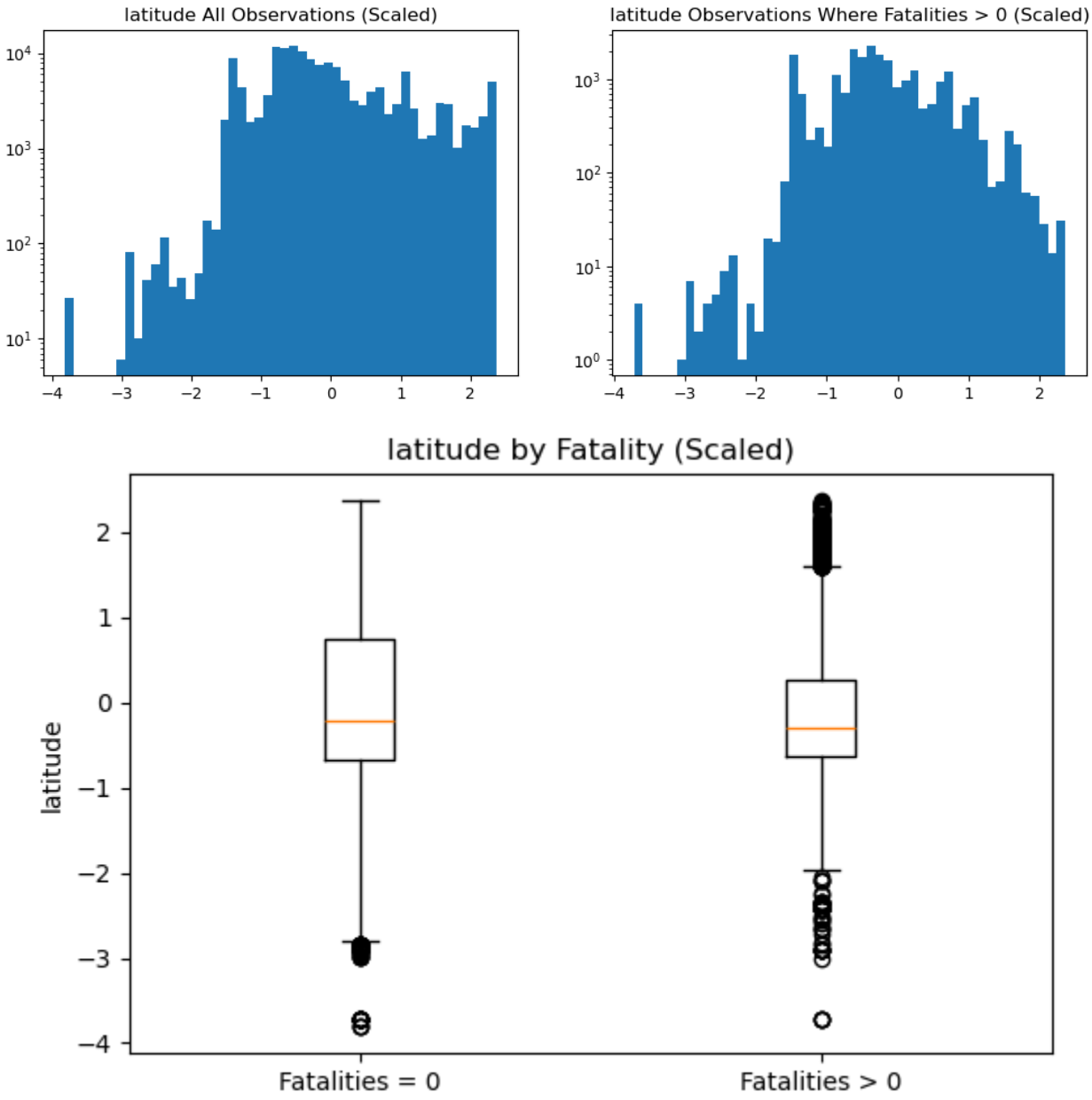
The Pearson correlation chart is conducted over all the independent variables we will be using to predict the target feature, fatalities. In this diagram, we divide the covariance of each variable by their standard deviations multiplied. This shows a correlation between –1 and 1 where 0 represents no correlation. The brighter sections indicate a higher correlation with the other variable and vice versa. We can notice that much of the equipment is highly correlated with each other. This can mean two things, either this type of equipment is always deployed at the same time, or it means that equipment is only documented rarely and when it is, multiple forms of equipment are documented at that time. Therefore, we may have sparse reporting of equipment in this dataset, which is plausible since it is hard to record exact numbers across such a large front for every single engagement.



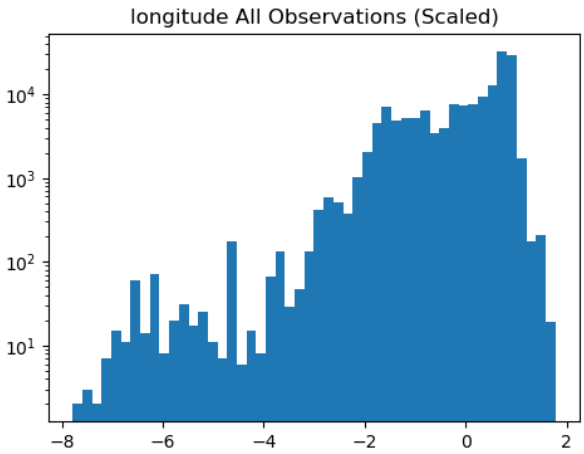
The first two principal components capture 0.64 and 0.12 of the data respectively and can explain about 76% of the variance in the data. The top 3 features for PCA 1 include drone, naval ship, and tank. The top 3 features for PCA 2 include military auto, POW, and fuel tank. This implies that most of the variance captured by the first component refers to combat situations and reports, whereas the variance captured by the second component refers to more logistical situations such as the capture of troops and refueling. In addition, we have overlayed the K means clustering regions based on k = 3. This algorithm works through clustering similar groups based on distance from a centroid point through unsupervised learning. We can see several distinct clusters, and k = 3 provides the cleanest separation for them. This indicates that our data can be differentiated and interpreted and that there are very different events in our dataset, but some common themes within clusters.

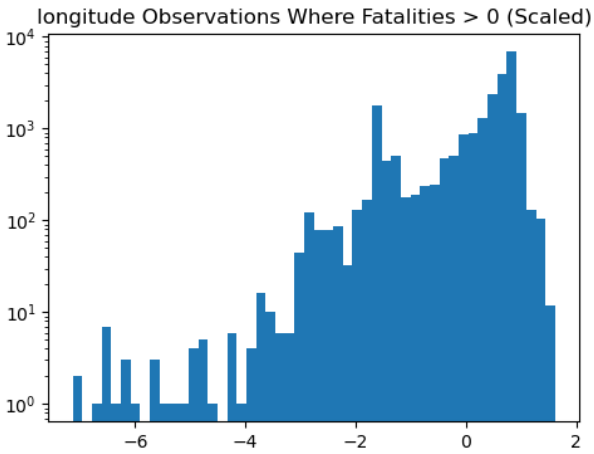


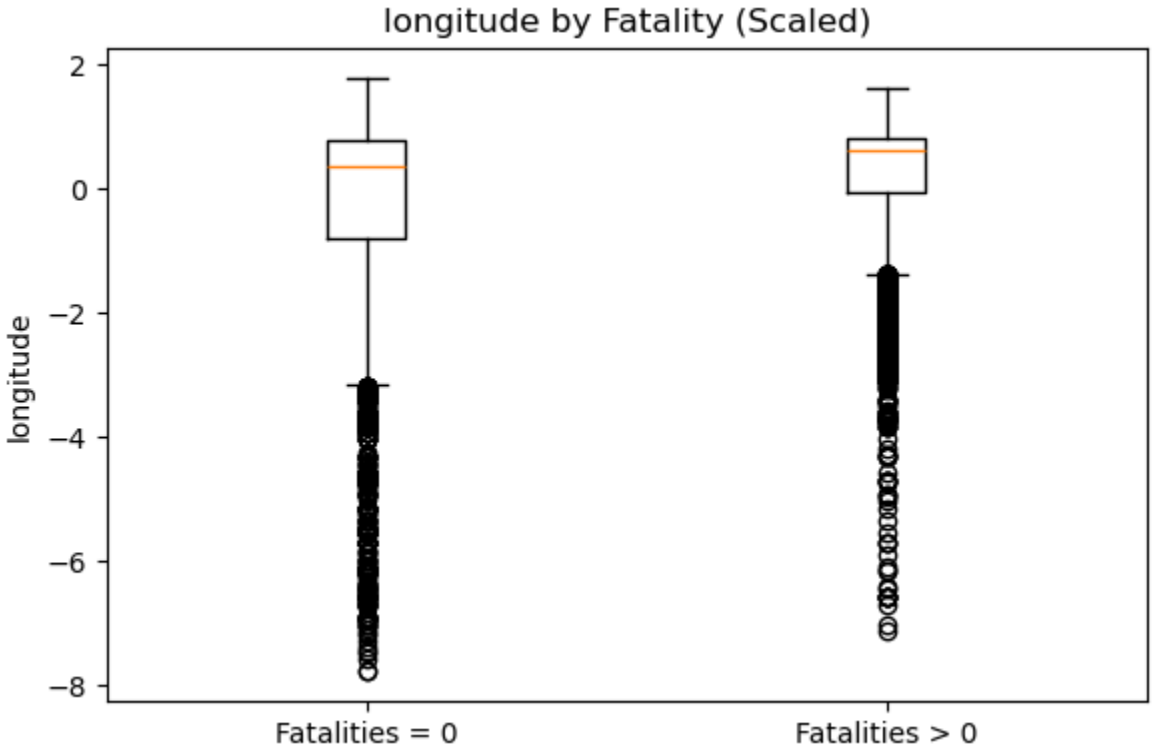
Most correlated feature with fatalities (log normalized and scaled). The charts show the distribution of the feature generally, as well as the distribution of the feature when fatalities are greater than 0. We can clip the outliers as we have a very normal distribution. This is a good variable because it is normal and very dense with few outliers and only a slight left skew when viewed with fatalities. It would make sense there are more fatalities where there are more people living.



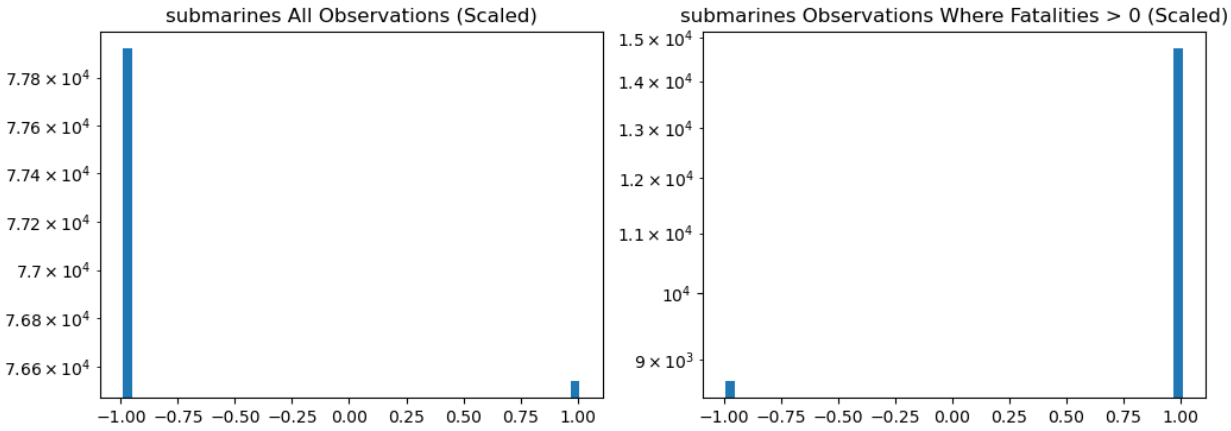
Second most correlated feature with fatalities (log normalized and scaled). The charts show the distribution of the feature generally, as well as the distribution of the feature when fatalities are greater than 0. Both latitude and longitude have a left skew across both distributions. This correlation makes sense considering the front is fixed to roughly one area in Ukraine. We are concerned about the outliers when fatalities > 0, which may indicate something interesting such as fatal rocket launches, drones, etc. We are still exploring these outliers.

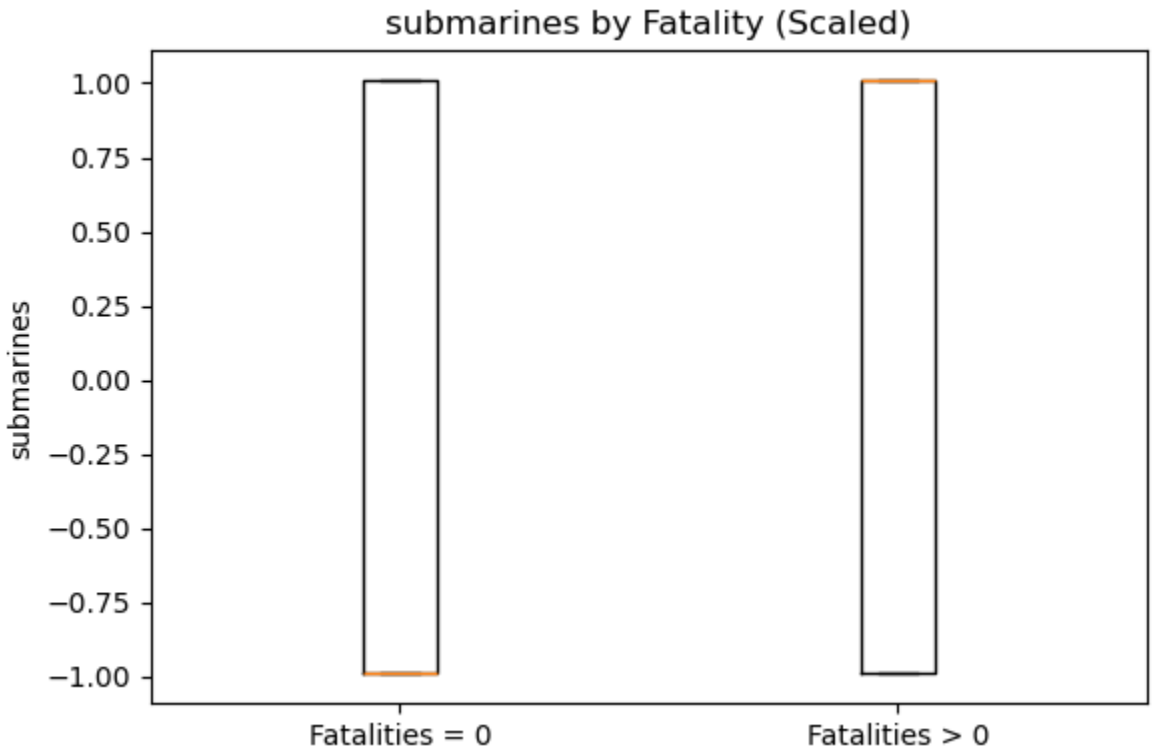




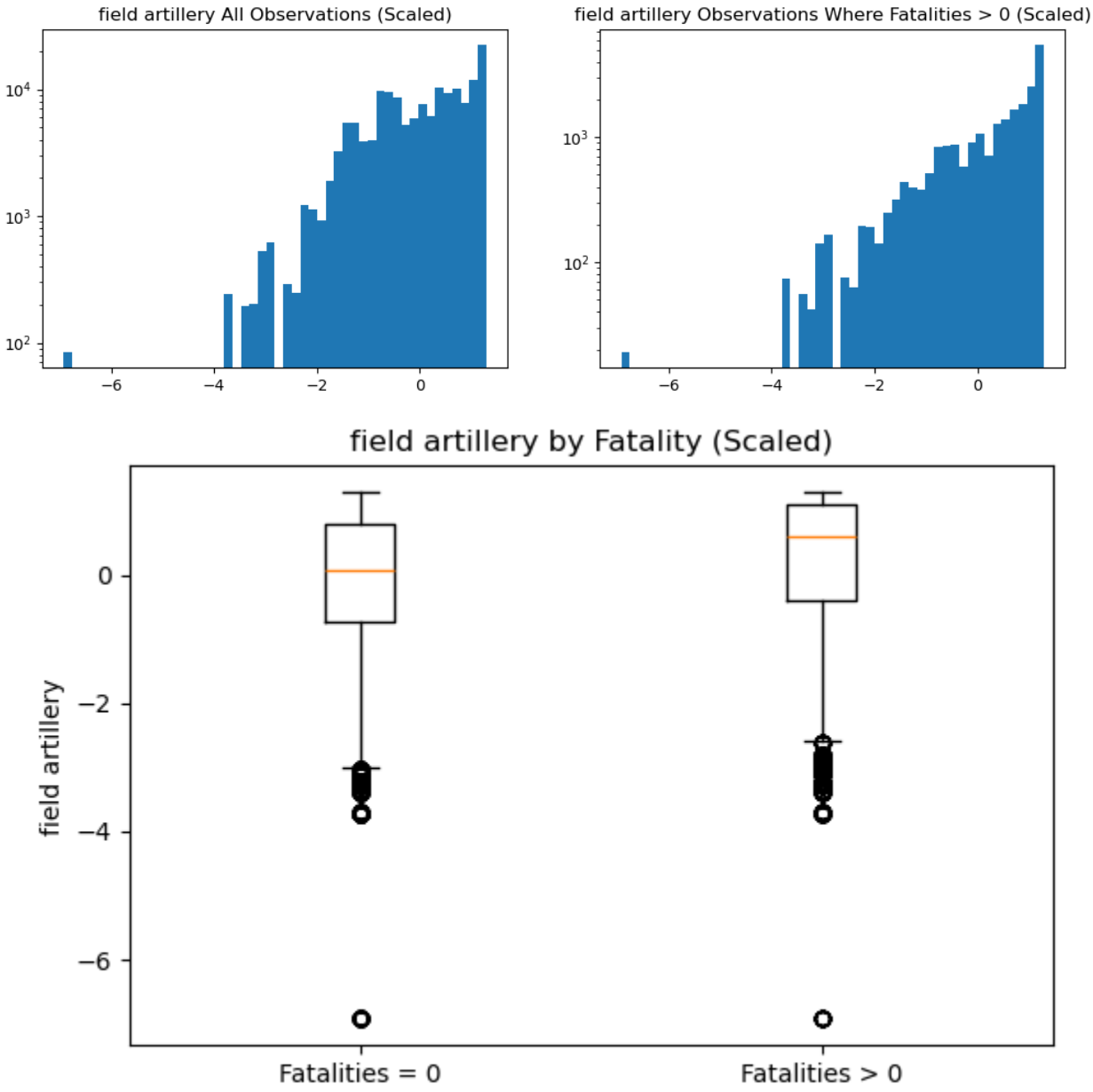


Third most correlated feature with fatalities (log normalized and scaled). The charts show the distribution of the feature generally, as well as the distribution of the feature when fatalities are greater than 0. Both latitude and longitude have a left skew across both distributions. This correlation makes sense considering the front is fixed to roughly one area in Ukraine. There is a problem with this data, as we seem to have a bimodal distribution, which indicates something strange about the data collection or that there are two main conflict zones.





Fourth most correlated feature with fatalities (log normalized and scaled). The charts show the distribution of the feature generally, as well as the distribution of the feature when fatalities are greater than 0. Submarines are extremely sparse, which makes sense considering they are invisible to most observers and radar systems. However, submarine deployment almost always results in high fatalities, so this is a very sparse, but strong indicator of fatalities. We may have to conduct synthetic oversampling on the data categories which are underreported via SMOTE or a similar technique.



Fifth most correlated feature with fatalities (log normalized and scaled). The charts show the distribution of the feature generally, as well as the distribution of the feature when fatalities are greater than 0. Field artillery is heavily left skewed because it rarely gets reported, and when it is reported, it is done in large numbers. This likely precedes a large engagement and can serve as a strong indicator for the target variable.

Exploration of Candidate Models

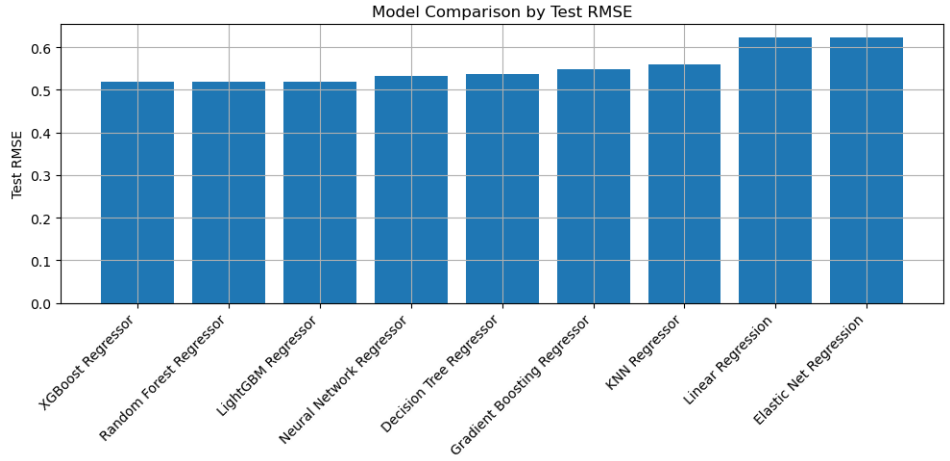
Our task is to predict a continuous value of fatalities given a set of predictors. Both the predictors and the target variable are sparse and imbalanced as we showed in our previous

analysis of the data. Given this constraint, we prioritize regression models which are well suited to working with imbalanced and sparse data. Our goal is to predict fatalities as a function of the nature of an independent event in the war. This is important as we are not predicting fatalities given previous trends, as would be done with a time series, rather we are predicting fatalities as a function of the properties of the event. Implementing this model will allow a commander to evaluate the scale of a military engagement given a particular set of parameters for the engagement.

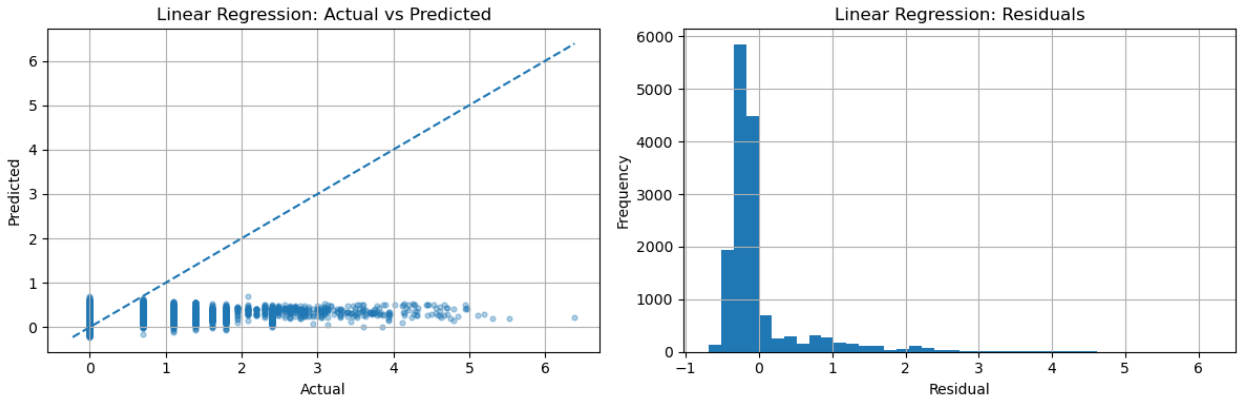
We initialize the preprocessed data and split it into 80% training, 10% validation, and 10% test, splitting on index. We log transform the target y variable to increase model performance by normalizing the distribution of the target. We explored 9 different models (untuned) and evaluated their performance below across the validation and test split using MSE, RMSE, MAE, and R2 scores. Our goal is to minimize RMSE, so we will select the model with the lowest score. In the following section, we explain each model and its implications in detail.

Model Performance Comparison

	model	val_MSE	val_RMSE	val_MAE	val_R2	test_MSE	test_RMSE	test_MAE	test_R2
0	XGBoost Regressor	0.263100	0.512933	0.276709	0.327474	0.269805	0.519428	0.279061	0.333978
1	Random Forest Regressor	0.264668	0.514459	0.246469	0.323467	0.270130	0.519740	0.249204	0.333178
2	LightGBM Regressor	0.262410	0.512260	0.277167	0.329238	0.270139	0.519749	0.279476	0.333154
3	Neural Network Regressor	0.273890	0.523345	0.287238	0.299893	0.283443	0.532393	0.290737	0.300314
4	Decision Tree Regressor	0.282867	0.531853	0.276305	0.276945	0.289049	0.537633	0.279441	0.286473
5	Gradient Boosting Regressor	0.293135	0.541420	0.308904	0.250699	0.301336	0.548941	0.310907	0.256143
6	KNN Regressor	0.307229	0.554283	0.260433	0.214672	0.314465	0.560772	0.262616	0.223733
7	Linear Regression	0.374760	0.612176	0.362046	0.042053	0.387021	0.622110	0.366102	0.044629
8	Elastic Net Regression	0.375396	0.612696	0.362710	0.040427	0.388013	0.622907	0.366796	0.042180

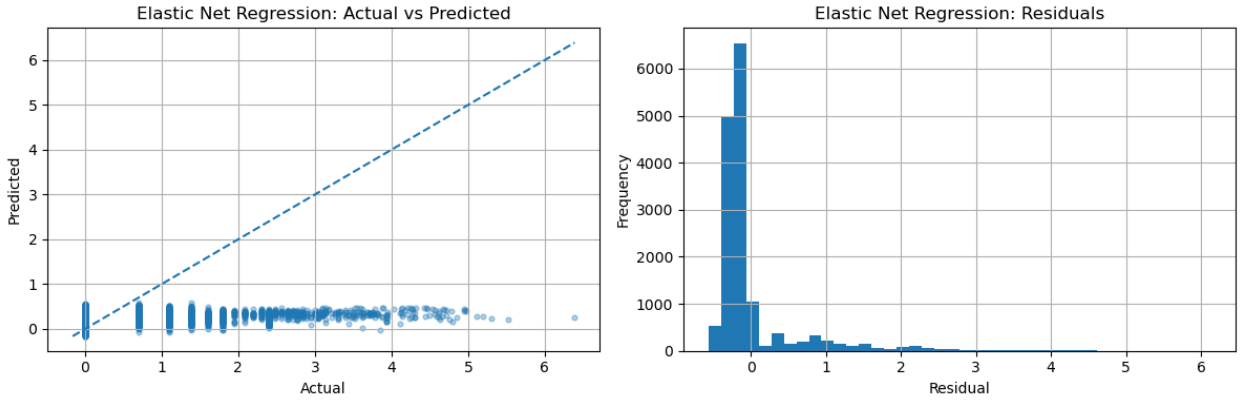


Linear Regression



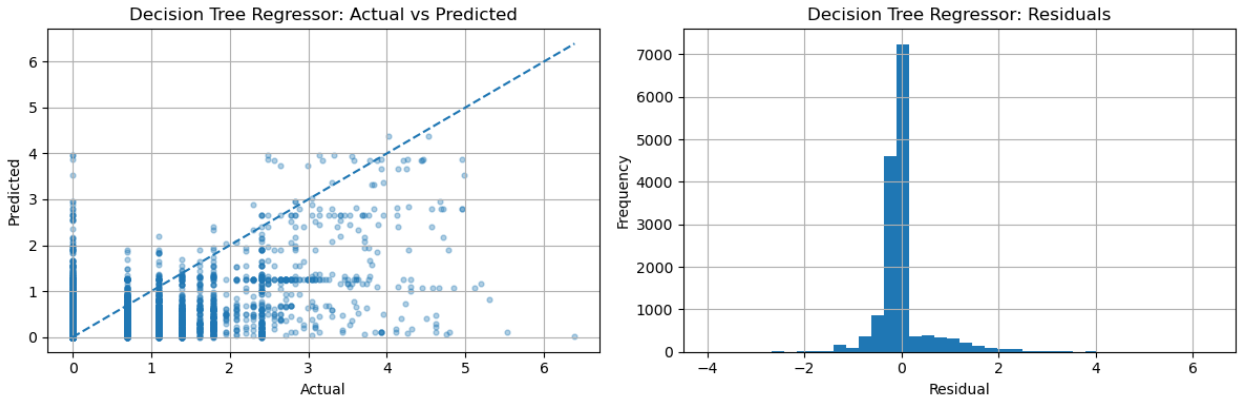
This model assumes that the data is not collinear and that it has a set of linearly independent predictors. With linear regression, we are minimizing the squared error of the predicted vs actual values. By applying to our data, we have a baseline model which shows major issues with the data. Most importantly, in the residuals, you will notice that estimates where 0 fatalities are reported are significantly wrong in all our model predictions. We believe this is a feature, not a bug; these are probably cases where the fatalities were not reported, but they probably did occur. Unfortunately, we cannot verify this as a fact, but this under-reporting will be a major issue moving forward for predictive modeling. This model struggles because events in this dataset are extremely polar, either a lot is recorded, or nothing at all is recorded.

Elastic Net Regression



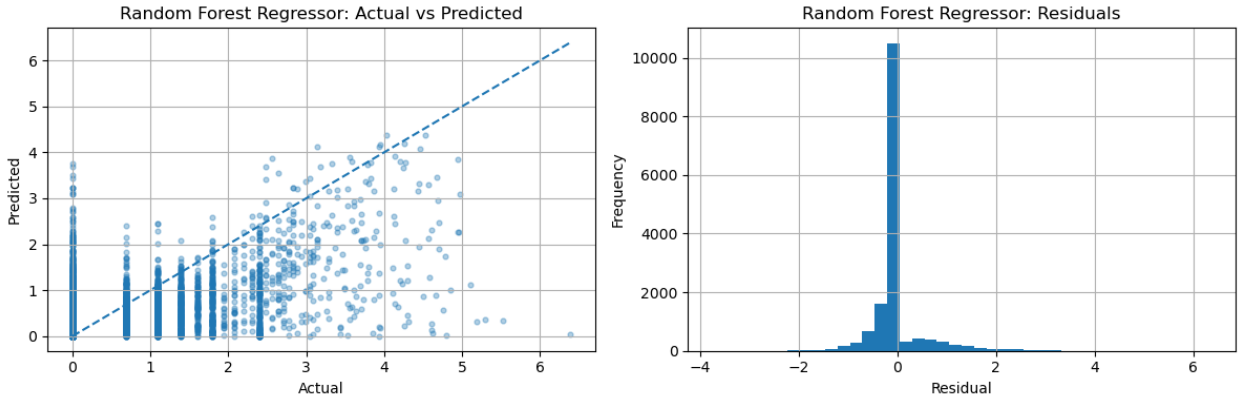
This variant of Linear Regression performs L1 and L2 regularization, which should help the model prevent overfitting in the case where we have many encoded variables. L1 regularization adds a penalty for the sum of absolute values of the weights to the loss function. L2 uses the squared weights as the penalty for the loss function. Combined, this should work better than base regression but still fails because of colinearity and non-linear relationships in the data.

Decision Tree Regression



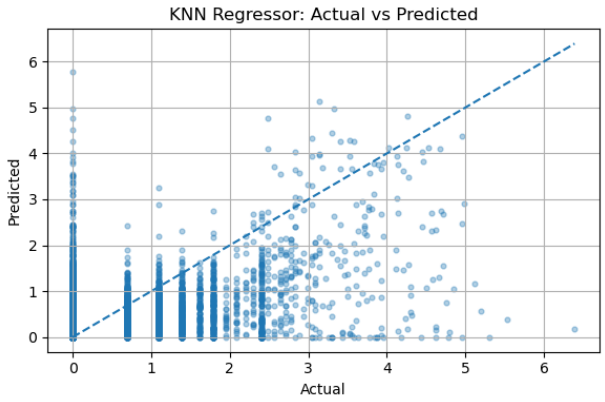
In the case of non-linear relationships, decision trees are a great choice for highly sparse, mostly categorical data. The base model shows promise, since the residuals are mostly normally distributed. Additionally, this model is closer to predicting true values than any of the other models so far. This model works though leaves and branches, where error is minimized by splitting the data into smaller partitions. In our scenario, this model works well because we have many categories encoded, and strong signals between equipment features in the data. This model allows the data to be partitioned into very niche situations with lower error rates on fatality prediction overall.

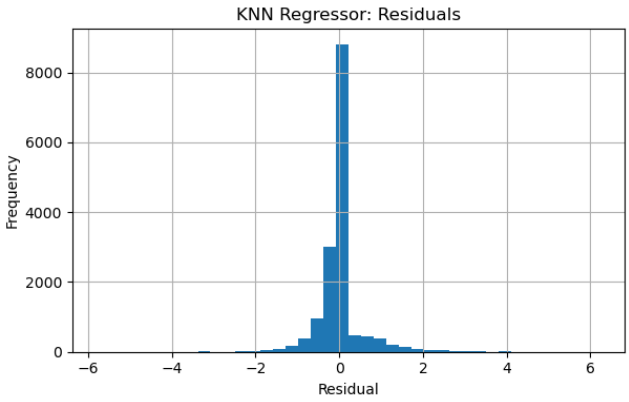
Random Forest Regression



Random Forests use the same logic in terms of decision tree construction, but they are a collection of decision trees, composing a forest. This ensemble method enables random partitioning of the data to construct various trees from partitions which include subsets of the data. This model should be less sensitive to overfitting than a regular decision tree and has more balanced predictions. Through random partitioning, it guarantees that it does not become over reliant on one feature. We tried this method because it produces more mean-centered normalized error and can work to generalize better than a single decision tree.

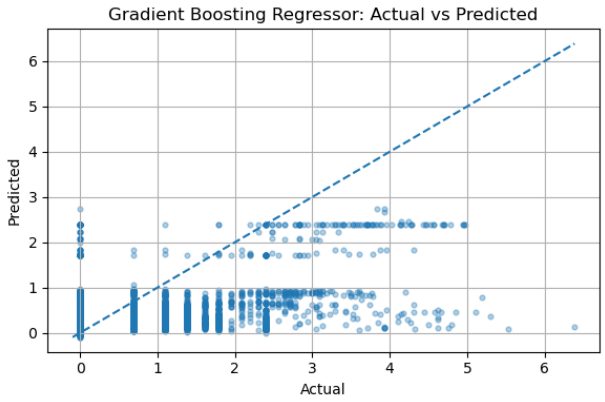
KNN Regression

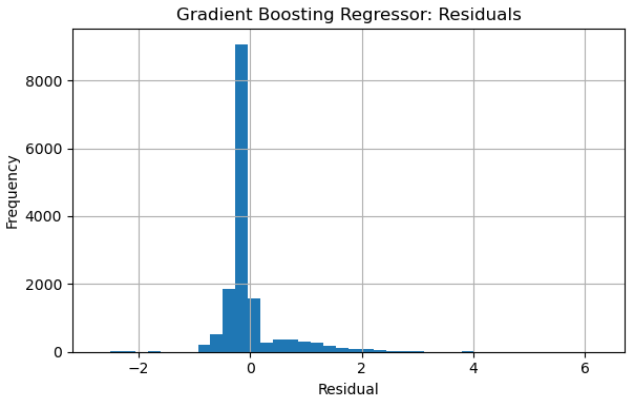




K Nearest Neighbors Regression uses similar instances to predict similar outcomes. So, for example if there are many cases which predict 0 fatalities and they are all very similar, then this model will predict 0. However, this model will not work well if the data is highly varied and there are very different neighbors with similar target values. This model works well if past events are similar to future events, but it struggles with our data because of the many dimensions we have. Given our large sparse features, the model still struggles as the tree-based models do.

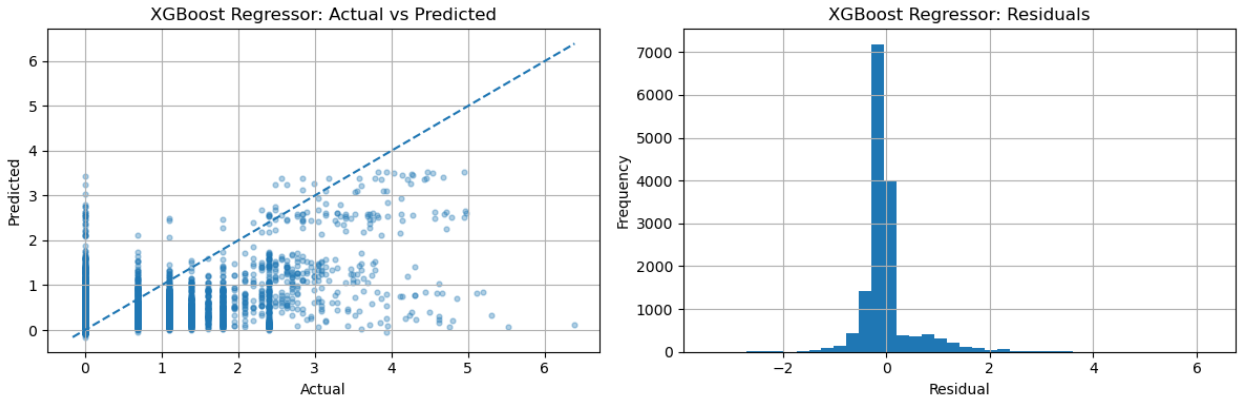
Gradient Boosting Regression





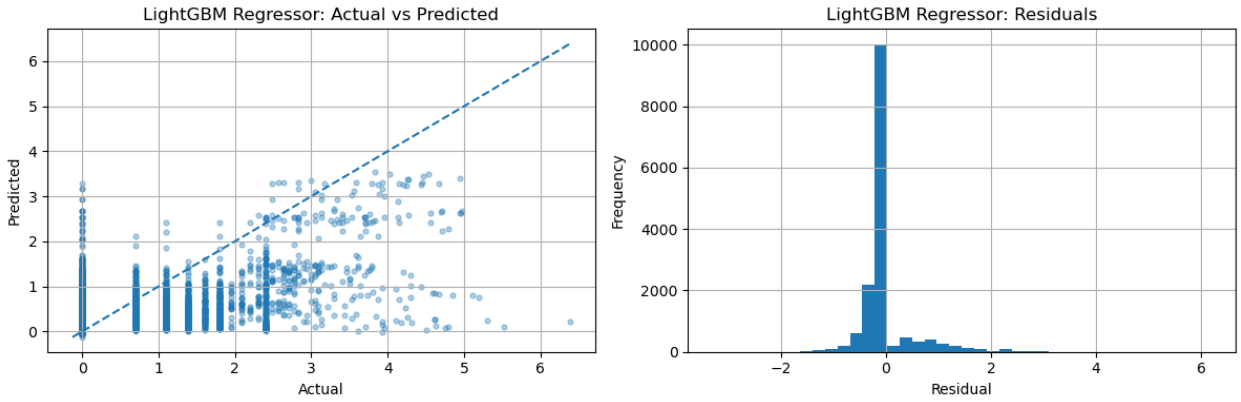
This ensemble method also uses decision trees, but instead of building decision trees from random partitions of the data, it builds sequential decision trees such that the next decision tree must further reduce the residual error of the previous decision tree. This optimization allows the model to learn complex patterns in the data and to rebalance the tree in the most effective way. This untuned model performs roughly as well as the other decision tree based algorithms. We notice residual error is normalized and centered around the mean. We tried this and other tree-based algorithms because they should work well for sparse and high dimensional data with a continuous target variable.

XGBoost Regression



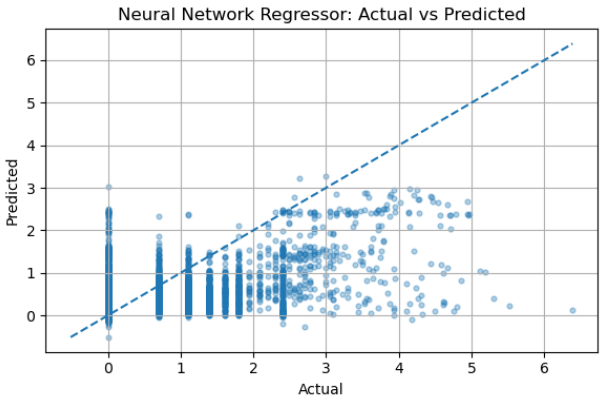
XGBoost involves building a whole sequence of decision trees with each of them trying to fix the errors from the previous one. It's usually good with non-linear relationships, mixed feature types and tabular data. It also happens to be bad with highly skewed targets and extreme outliers This XGBoost model had a positive trend from 0 to 2.5 under predicts beyond that. The sharp spike near 0 on the residuals indicate how the model could be over relying on just a handful of features for the predictions.

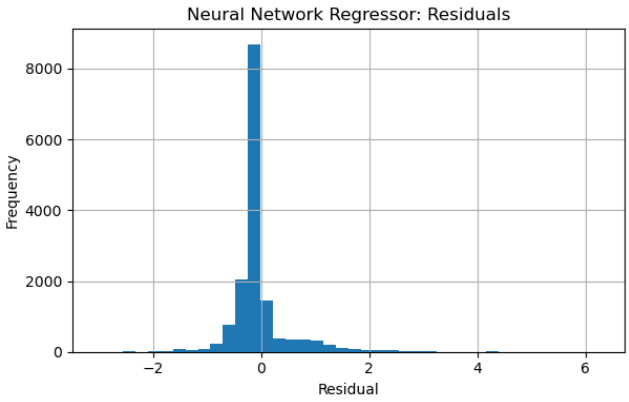
LightGBM Regression

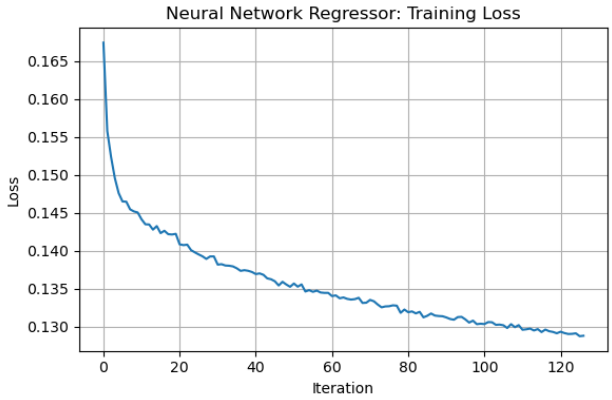


LightGBM is essentially the same as XGBoost but it trains faster and more efficiently. Other than that, the two methods perform very similarly. The LightGBM Regressor's predictions performed best from 0 to 2.5. Like the other models we've tried, it struggles with predicting the higher values. It only predicts as high as somewhere between 3 and 4 but the actual numbers go as high as 6. The residuals spike at near 0 with a right tail. This could mean that the model was heavily relying on only a few dominant features.

Neural Network Regression

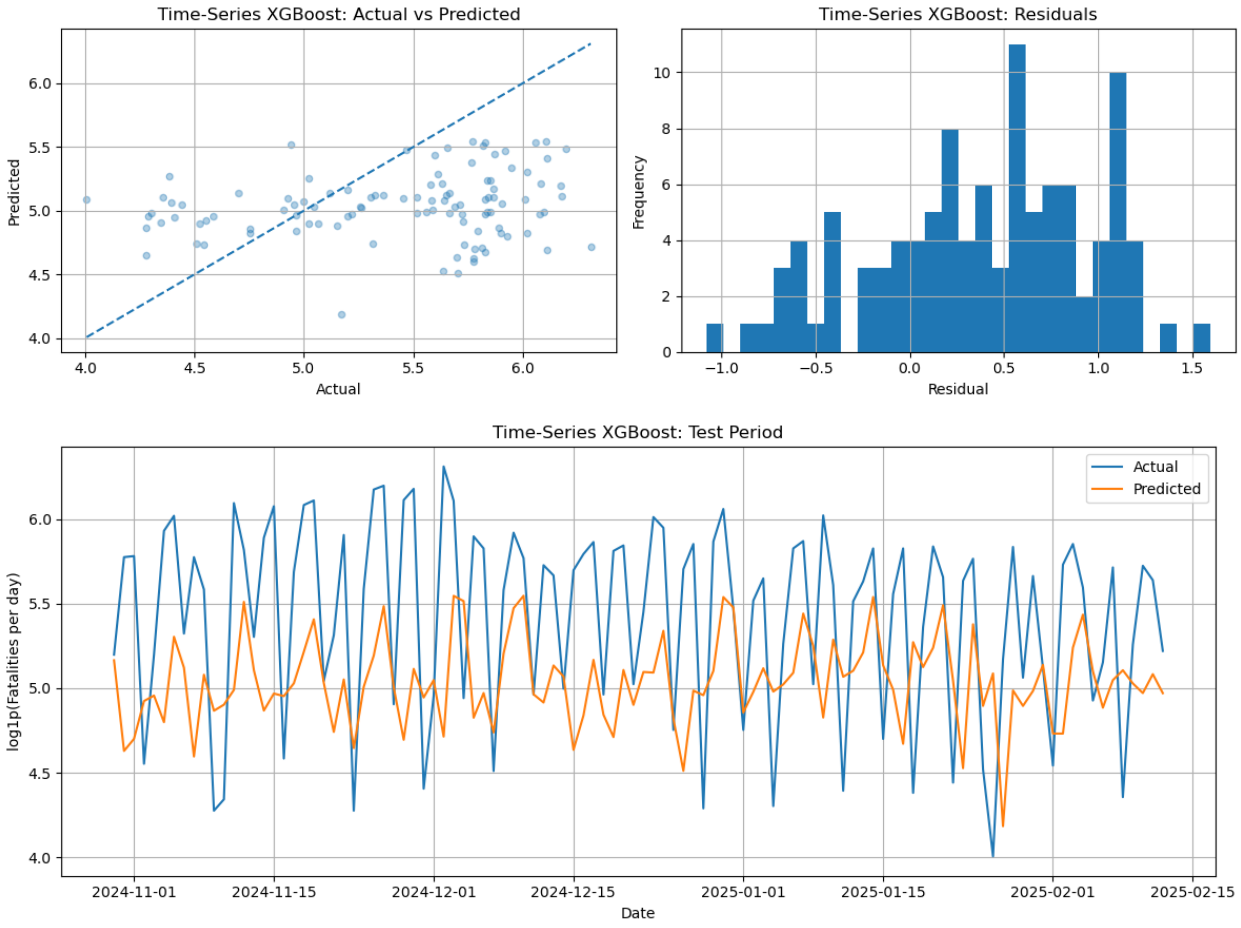






As the name suggests, neural network regressors involve nodes that form the input layer, the hidden layers and the output layer. After making each prediction, it adjusts each weight according to how much each of them contributed to that error in a process called backpropagation. Unlike the previous two models, this one tries to learn continuous relationships between the features and the target variable. This would make it better at capturing complex relationships which is why this model performed slightly better than other previous models. Unfortunately, neural networks need a ton of training data and tuning to do its job well. This Neural Network Regressor model had a training loss that fell sharply from 0.165 to less than 0.13 over 125 iterations. Chances are the model won't benefit much beyond that point. Unfortunately, the model still underpredicts with a maximum of 3 so it's unable to track the actual values beyond that threshold. The residual distribution spikes near 0 with a tail that extends to around 6 which reinforces how much the model is underpredicting. Overall, it performs very similary to the LightGBM Regressor.

Time Series XGBoost Demo (with no future peeking)




```
Test metrics:
MSE      0.474973
RMSE     0.689183
MAE      0.586255
R2      -0.481586
dtype: float64
```

XGBoost without future peeking is XGBoost but with a fixed cutoff date that ensures the model isn't training on data that came after that cutoff date. That way, the model isn't seeing any future outcomes from its training. More importantly, tuning the model this way would allow us to conclusively decide if we should stick with framing this as a time series problem. That being said, this XGBoost model does a decent job of capturing the general level of fatalities when they're on a log scale. Unfortunately, it underpredicts the peaks from the actual data and the volatility is less. The R^2 of -0.48 confirms that the XGBoost model is worse than a mean baseline. The residual distribution is skewed to the right which makes sense with the consistent underprediction from the scatter plot. The XGBoost model is simply not capable of capturing the day-to-day variance so we can stop looking at this challenge from a time-series perspective.

Best Model Performance Evaluation

Previously, we evaluated multiple models on the task of regression to predict a continuous target of fatalities. Now, we reframed from a regression task to a classification task, where we are now building a model to accurately classify fatalities within a range of 0-1, 1-25, 25-50, etc. We chose this approach because of the extreme scarcity of rare events in the data where there are high fatalities, making it almost impossible to model exact fatalities without overfitting. Our model now predicts with 86% accuracy within 6 fatality ranges, correctly classifying common events and less accurately classifying rare events. We are not interested in knowing the exact fatality count, but rather if they are predicted to occur at all, and if so, roughly how many. This model achieves our goal of predicting conflict casualties better than the regression model because we have now binned our prediction ranges, driving accuracy higher without compromising the overall intention of our modeling. We are selecting our final model with accuracy as the key metric because we believe it is equally important for the model to the majority class (0 fatalities) as it would be for it to classify any other fatality range. For a commander in the battlefield, a 0-fatality prediction will mean that the battle is considered low risk in terms of personnel losses, or that the type of military action being planned is covert enough to avoid reporters documenting losses.

Below, we have summarized our findings of the best three models, and find that they are very similar, but we select XGBoost since it performed slightly better on the regression task, but we could have picked any of the three models.

Scores for all three models are roughly the same across all categories; since they are all variants of tree-based algorithms this makes sense. XGBoost performs the best at balancing classification of rare events and common events overall.

XGBOOST

XGBoost — Classification Report

	precision	recall	f1-score	support
0	0.88	0.98	0.93	26205
1-25	0.61	0.20	0.30	4431
25-50	0.38	0.16	0.23	142
50-150	0.42	0.27	0.33	100
150-300	0.00	0.00	0.00	12
300+	0.00	0.00	0.00	3
accuracy			0.86	30893
macro avg	0.38	0.27	0.30	30893
weighted avg	0.83	0.86	0.83	30893

Random Forest

Random Forest — Classification Report

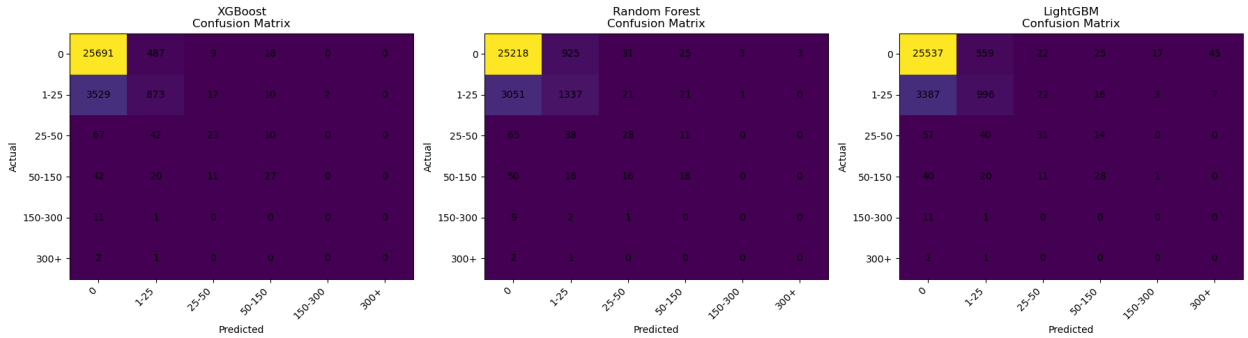
	precision	recall	f1-score	support
0	0.89	0.96	0.92	26205
1-25	0.58	0.30	0.40	4431
25-50	0.29	0.20	0.23	142
50-150	0.24	0.18	0.21	100
150-300	0.00	0.00	0.00	12
300+	0.00	0.00	0.00	3
accuracy			0.86	30893
macro avg	0.33	0.27	0.29	30893
weighted avg	0.84	0.86	0.84	30893

LightGBM

LightGBM — Classification Report

	precision	recall	f1-score	support
0	0.88	0.97	0.92	26205
1-25	0.62	0.22	0.33	4431
25-50	0.36	0.22	0.27	142
50-150	0.34	0.28	0.31	100
150-300	0.00	0.00	0.00	12
300+	0.00	0.00	0.00	3
accuracy			0.86	30893
macro avg	0.37	0.28	0.31	30893
weighted avg	0.84	0.86	0.83	30893

Confusion Matrix for all three models shows how imbalanced the data is. For a better understanding of the models, the ROC charts show more clearly per-class classification.



Class counts from smallest to largest fatality range (most fatalities recorded are 0)

Training data

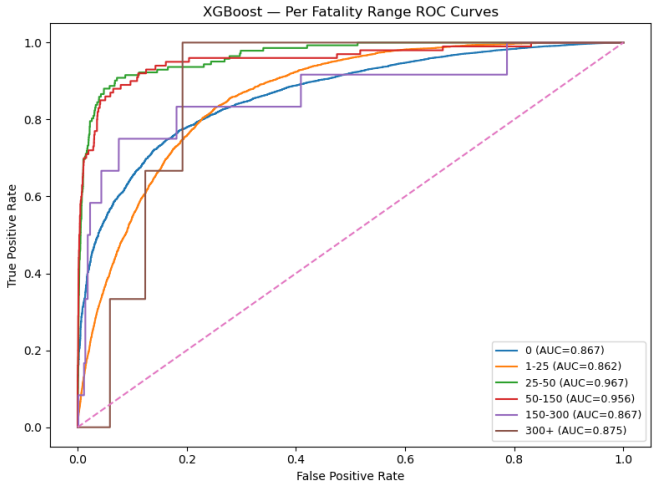
0	104820
1	17724
2	568
3	398
4	47
5	12

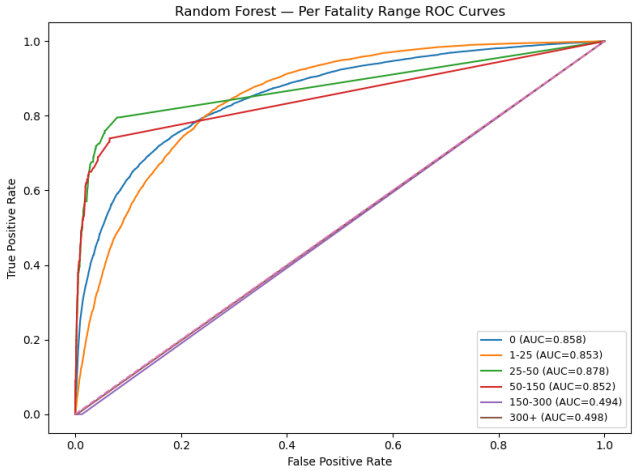
Test data

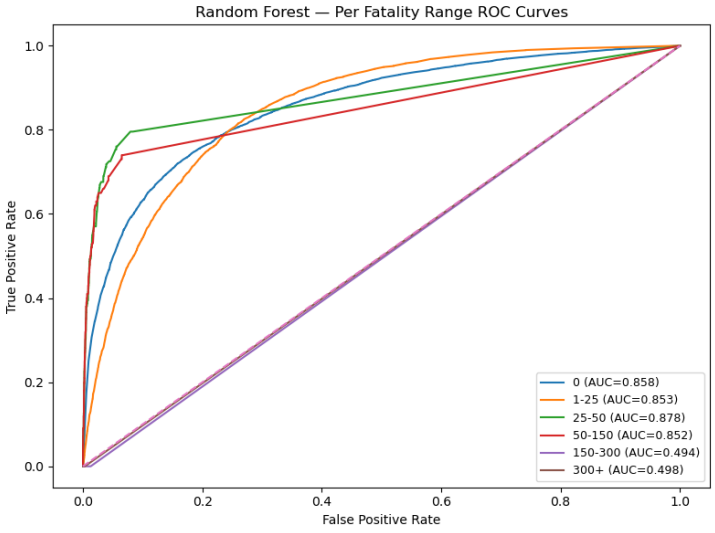
0	26205
1	4431
2	142
3	100
4	12
5	3

	Model	Accuracy
0	Random Forest	0.8611
1	LightGBM	0.8608
2	XGBoost	0.8615

Here, you can see why we select XGBoost most clearly. The AUC for each class is much higher than for the other classifiers. Additionally, it appears more consistent to predict on rare fatality counts (large values) than the other models


XGBoost — Per Fatality Range ROC Curves
True Positive Rate
False Positive Rate
0 (AUC=0.867)
1-25 (AUC=0.862)
25-50 (AUC=0.967)
50-150 (AUC=0.956)
150-300 (AUC=0.867)
300+ (AUC=0.875)


Random Forest — Per Fatality Range ROC Curves
True Positive Rate
False Positive Rate
0 (AUC=0.858)
1-25 (AUC=0.853)
25-50 (AUC=0.878)
50-150 (AUC=0.852)
150-300 (AUC=0.494)
300+ (AUC=0.498)


Random Forest — Per Fatality Range ROC Curves
True Positive Rate
False Positive Rate
0 (AUC=0.858)
1-25 (AUC=0.853)
25-50 (AUC=0.878)
50-150 (AUC=0.852)
150-300 (AUC=0.494)
300+ (AUC=0.498)

Conclusion and Key Takeaways

For the small Stone, we reframed the original regression problem into a classification challenge by binning the fatalities into numerical ranges. This change single-handedly helped us handle the imbalance in the data. Set a lot better, specifically the scarcity of events with high fatalities, which resulted in major improvements in our models. Our XGBoost, random Forest and light GBM models performed pretty much the same across our evaluation metrics. This made sense given that they're all similar tree-based models. We ended up picking XGBoost because it had slightly higher performance than the others, according to its AUC scores. The overall accuracy of 86% demonstrates how much our models have improved since the beginning of this project, but we do have to acknowledge that a lot of this performance comes from the prevalence of events where there were no fatalities. Nonetheless, the model was still able to come up with substantial predictions across different ranges of fatalities. This ultimately makes it useful for high-level decision making. Overall, this step of our project is a great example of how various approaches to a data science problem can yield drastically different results in model development. Our final model would align well with real world use cases where military leadership would rather look at approximations of results than exact figures.